# Assignment 2: From Trees to Neural Networks

COMS 4995 — Applied Machine Learning

Anh Lam (adl2196)

March 2026

## Introduction

This report presents my exploration and comparison of two powerful modeling paradigms—**Gradient Boosted Decision Trees (GBDT)** using XGBoost and **Multi-Layer Perceptrons (MLP)** using scikit-learn—on the *Home Credit Default Risk* dataset from Kaggle.

**GitHub Repository:** `https://github.com/amelia751/coms4995-assignment2`

**Problem Definition.** The task is binary classification: predict whether a loan applicant will repay their loan (TARGET = 0) or default (TARGET = 1). This is a critical task in consumer finance, where accurate risk assessment reduces losses while ensuring creditworthy applicants receive loans.

**Dataset Description.** The dataset contains 307,511 loan applications with 121 features, including demographic information (age, gender, family status), financial metrics (income, credit amount, annuity), external risk scores, and housing characteristics. The target variable is highly imbalanced—only 8.07% of applicants defaulted—making this a challenging classification problem where naive models can achieve high accuracy by simply predicting the majority class.

## 1 Data Preparation

**Missing Values.** I identified several data quality issues: (1) `DAYS_EMPLOYED` contained the anomalous value 365,243 for 55,374 retired/unemployed applicants—replaced with NaN; (2) `CODE_GENDER` and `ORGANIZATION_TYPE` contained "XNA" placeholder values—treated as missing. Columns with >40% missing values were dropped entirely (e.g., building characteristics with ∼70% missing), as imputation would introduce excessive noise. Remaining missing values were imputed using the **median** strategy, fit on the training set only to prevent data leakage.

**Feature Engineering.** I constructed 9 new features to capture domain-relevant relationships:

- **Credit-to-income ratio**, **annuity-to-income ratio**, and **credit-to-goods-price ratio**—capturing leverage and repayment burden
- **Age in years** and **employment years**—derived from negative day counts
- **Age group**—discretized age for non-linear effects
- **Income per child**—household financial burden proxy
- **External source mean and std**—aggregating the three EXT_SOURCE features, which are among the most predictive

**Encoding.** Binary categorical features (e.g., FLAG_OWN_CAR) were label-encoded. Multi-class categoricals (e.g., ORGANIZATION_TYPE with 57 unique values) were one-hot encoded with `drop_first=True` to avoid multicollinearity.

**Train/Validation/Test Split.** Data was split 70/15/15 with stratification on the target to preserve the class imbalance ratio across all sets. The resulting training set contained 215,257 samples, validation 46,127, and test 46,127.

**Feature Scaling.** For MLP, all features were standardized using `StandardScaler` (zero mean, unit variance), fit on the training set only. This is necessary because neural networks use gradient-based optimization where features on different scales cause uneven gradient magnitudes, leading to slow or poor convergence. Tree-based models like XGBoost are invariant to monotonic feature transformations—they split on thresholds, so scaling has no effect.

# 2 Gradient Boosted Decision Tree (GBDT)

I trained an `XGBClassifier` with the following approach:

- **Class imbalance**: Used `scale_pos_weight` = 11.39 (ratio of negatives to positives) to penalize misclassification of defaults more heavily
- **Early stopping**: Trained with up to 1,000 boosting rounds but used `early_stopping_rounds=50` on validation log-loss to prevent overfitting
- **Baseline parameters**: learning_rate=0.1, max_depth=6, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1, reg_lambda=1.0

I systematically explored: (1) learning rates {0.01, 0.1, 0.3}; (2) max_depth {3, 5, 7, 9}; (3) subsample {0.5, 0.7, 0.8, 1.0}; (4) regularization combinations of reg_alpha and reg_lambda.

## 2.1 Training vs. Validation Loss

The baseline XGBoost model converged at iteration 215 (early stopping from 1,000 max). Figure 1 shows training vs. validation loss—the gap between curves indicates some overfitting, but early stopping prevents the model from degrading further.

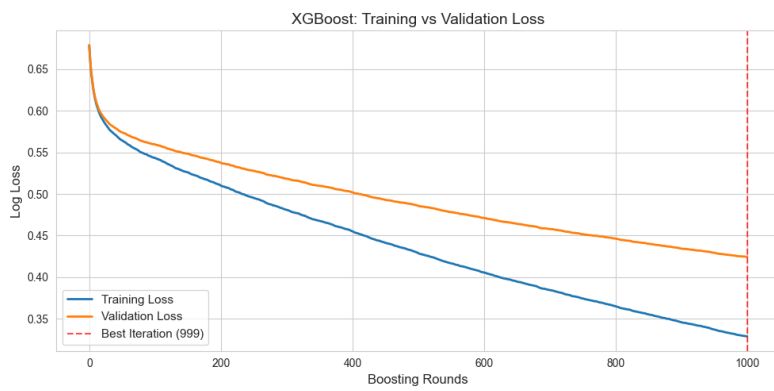


Figure 1: XGBoost training vs. validation log-loss with early stopping.

## 2.2 Feature Importance

Figure 2 reveals that the three `EXT_SOURCE` features dominate both weight-based and gain-based importance. My engineered features (EXT_SOURCE_MEAN, CREDIT_INCOME_RATIO) also rank highly, validating the feature engineering choices.

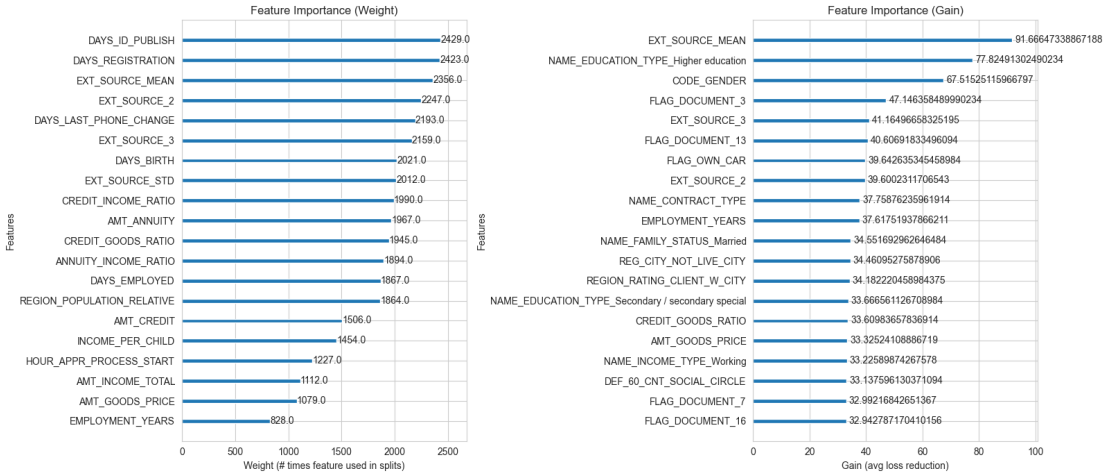


Figure 2: XGBoost feature importance by weight (left) and gain (right).

### 2.3 Effect of Learning Rate

LR=0.01 requires more iterations (605) but achieves the best validation loss; LR=0.1 balances speed and performance (215 iterations); LR=0.3 converges fastest (107 iterations) but overshoots slightly. See Figure 3.

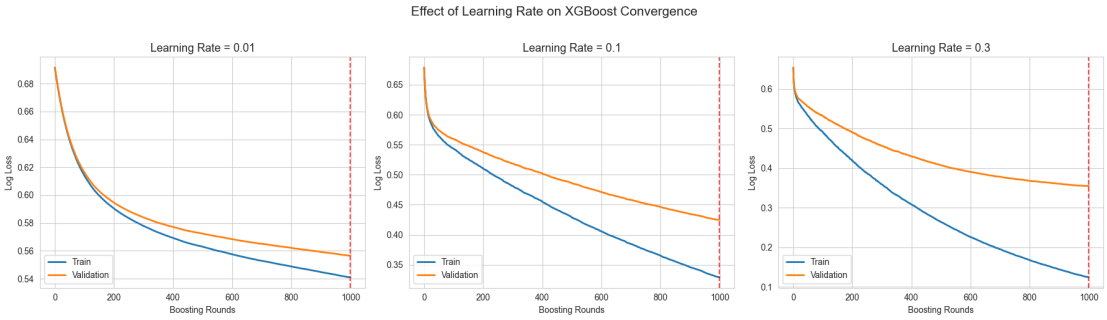


Figure 3: Effect of learning rate on XGBoost convergence.

## 3 Multi-Layer Perceptron (MLP)

I trained an `MLPClassifier` with the following approach:

- **Baseline architecture**: (128, 64) with ReLU activation
- **Early stopping**: validation_fraction=0.15, n_iter_no_change=15
- **Optimizer**: Adam with learning_rate_init=0.001

I explored: (1) architectures {(64,), (128,64), (256,128,64), (128,128), (256,128)}; (2) activations {relu, tanh}; (3) learning rates {0.001, 0.01, 0.1}.

### 3.1 Training Loss Curve

The baseline MLP (128, 64) converged in approximately 60 iterations. Figure 4 shows the training loss curve with the corresponding validation score.

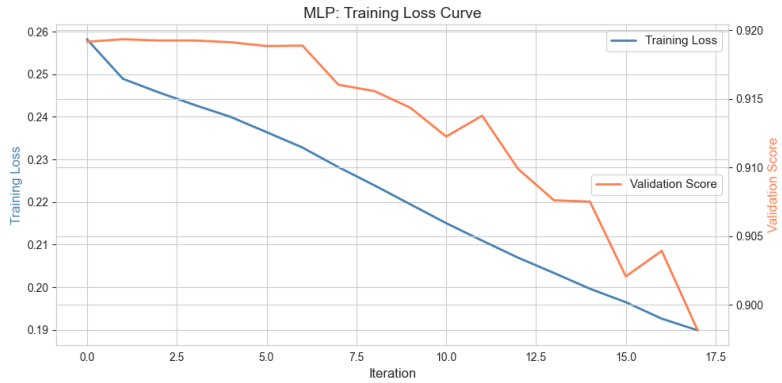


Figure 4: MLP training loss curve with validation score.

### 3.2 Effect of Network Depth/Width

Deeper/wider networks (256,128,64) achieve lower training loss but do not consistently improve validation performance, suggesting diminishing returns from increased capacity. ReLU and tanh achieved similar validation AUC-PR, though ReLU converged slightly faster. Learning rate 0.001 provided the most stable convergence; 0.1 caused unstable training. See Figure 5.

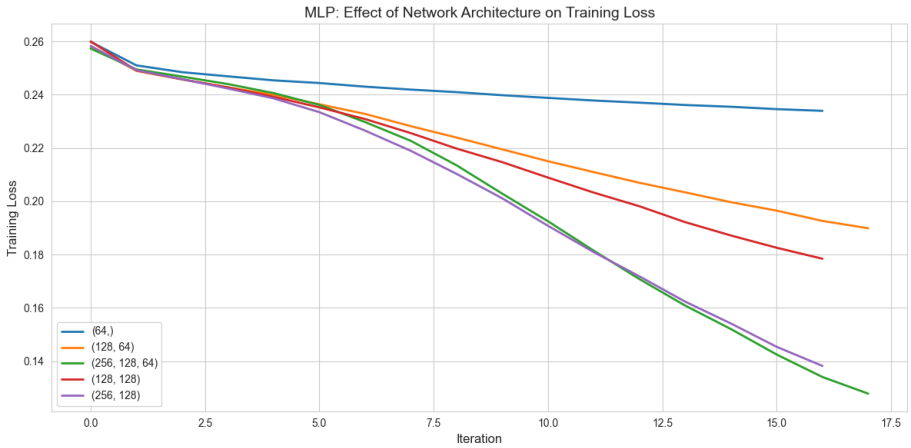


Figure 5: Effect of network architecture on MLP training loss.

## 4 GBDT vs MLP Comparison

### 4.1 Quantitative Comparison

Table 1 presents the side-by-side test set performance. Both models achieve comparable AUC-PR ($\sim$0.22) and ROC-AUC ($\sim$0.74), indicating similar ranking ability. However, their behavior at the default classification threshold (0.5) differs dramatically.

Table 1: GBDT vs MLP — Test Set Performance Comparison

| **Metric** | **XGBoost (GBDT)** | **MLP** |
|---|---|---|
| Accuracy | 0.8061 | 0.9192 |
| Precision | 0.2018 | 0.4419 |
| Recall | 0.4742 | 0.0051 |
| F1-Score | 0.2831 | 0.0101 |
| AUC-PR | 0.2198 | 0.2179 |
| ROC-AUC | 0.7374 | 0.7422 |
| Training Time (s) | 31.04 | 13.69 |

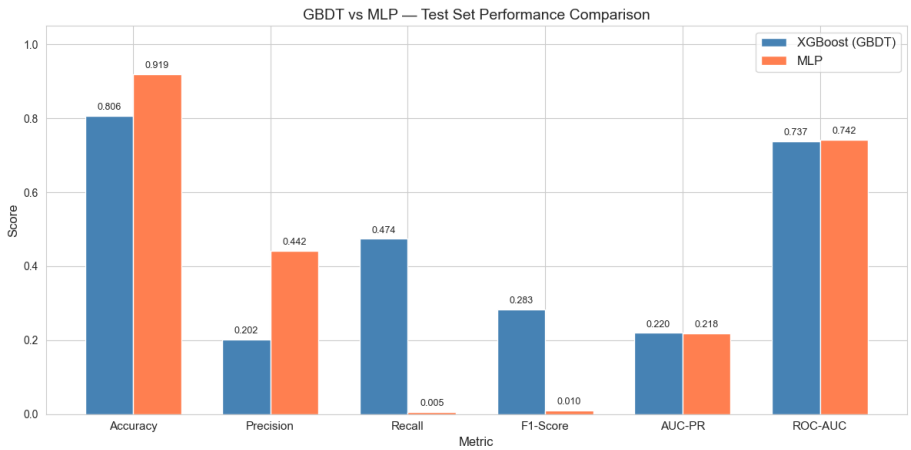


Figure 6: Bar chart comparison of GBDT vs MLP on test metrics.

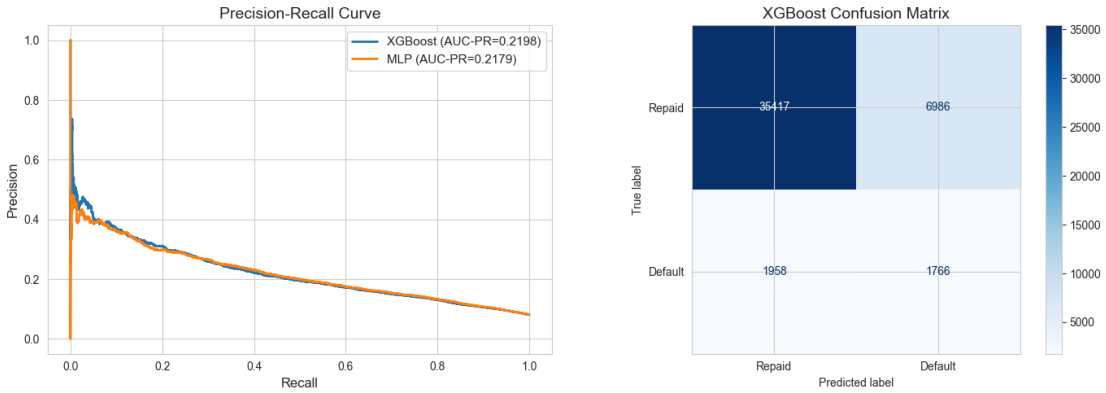


Figure 7: Precision-Recall curves (left) and XGBoost confusion matrix (right).

## 4.2 Discussion

**Key Observation: Class Imbalance Handling.** The most striking difference is in recall for the default class. XGBoost achieves 47.4% recall thanks to `scale_pos_weight`, which explicitly compensates for imbalance during training. MLP's recall is near-zero (0.51%) because `MLPClassifier` lacks a built-in class weighting mechanism, causing it to overwhelmingly predict the majority class. MLP's high accuracy (91.9%) is misleading—it achieves this by predicting "repaid" for almost every applicant.

Despite this threshold-level difference, the **AUC-PR and ROC-AUC are nearly identical** between the two models, showing that both learn comparable probability rankings. The difference lies in calibration and threshold selection, not in discriminative power.

**When to prefer GBDT over MLP:**

- **Tabular data** with mixed feature types—GBDT naturally handles numerical and categorical features
- **Class imbalance**—XGBoost's `scale_pos_weight` provides a simple, effective solution
- **Interpretability requirements**—feature importance is built-in
- **Minimal preprocessing**—handles missing values natively and is scale-invariant

**When to prefer MLP:**

- **High-dimensional dense features** (e.g., embeddings, image features)
- **Complex non-linear interactions** that benefit from learned representations
- **Multi-modal or multi-task** settings where neural architectures are more flexible
- When combined with proper class-weighting via **sample reweighting** or **threshold tuning**

**Interpretability.** GBDT provides direct feature importance (weight, gain, cover), enabling domain experts to validate that the model relies on sensible features (e.g., EXT_SOURCE scores, credit ratios). MLP is a black box—while post-hoc methods like SHAP exist, they add complexity and computational cost.

**Hyperparameter Sensitivity.** MLP is significantly more sensitive to hyperparameter choices. Learning rate changes from 0.001 to 0.1 can destabilize training entirely, and missing standardization renders the model non-functional. XGBoost with reasonable defaults (LR=0.1, depth=6, subsample=0.8) produces strong baselines with minimal tuning.

**Limitations.** (1) I only used the main application table; incorporating bureau and credit card history could improve both models. (2) MLPClassifier lacks built-in class weighting—using PyTorch with weighted BCE loss or SMOTE would likely improve MLP's recall. (3) Grid-based search was used; Bayesian optimization might find better configurations.

**Bias-Variance Tradeoff.** XGBoost with `scale_pos_weight` operates in a higher-variance regime (lower accuracy, higher recall), while MLP defaults to a high-bias regime on the minority class. XGBoost's widening train-val gap is mitigated by early stopping; MLP converges smoothly but fails to capture minority-class signal.

**Conclusion.** For tabular credit risk data, XGBoost is the more practical choice—competitive discriminative performance (ROC-AUC = 0.737), built-in imbalance handling, interpretability, and minimal preprocessing. MLP achieves similar ranking performance (ROC-AUC = 0.742) but needs more engineering for deployment.

## AI Tool Usage Disclosure

**AI tools used:**

- **Claude (Cursor AI assistant):** Used to generate boilerplate code templates (e.g., import blocks, plotting scaffolds, LaTeX document structure). The AI provided starter code snippets that were then modified and integrated into the final pipeline.

**My personal contributions:**

- Dataset selection (Home Credit Default Risk) and Kaggle environment setup
- Exploratory data analysis: examining distributions, missing value patterns, and class imbalance

- All data preprocessing decisions: choosing which columns to drop (>40% missing threshold), identifying anomalous values (DAYS_EMPLOYED = 365243, XNA entries), and selecting median imputation
- Feature engineering design: deciding which ratios, aggregates, and derived features to create based on domain understanding of credit risk
- Hyperparameter selection and tuning strategy for both XGBoost and MLP
- Interpreting all experimental results: analyzing learning rate trade-offs, architecture effects, regularization impact, and the class imbalance dynamics between models
- Writing all analysis and discussion in the report, including the bias-variance interpretation and model comparison insights
- Verifying no data leakage in the preprocessing pipeline (fit on train only)
- Reviewing, debugging, and validating all code logic and model configurations